

SEDust: User Manual

Abstract

This manual describes the model-agnostic dust thermal-emission library under SEDust/sed/. The library computes the infrared emission spectrum of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type and one emission call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

Contents

1. Overview	2
2. Building the library	2
3. Models and builders	2
4. Computing emission	3
4.1 Single-cell exact solve	3
4.2 Solver choice (m%stoch_method)	3
4.3 Equilibrium-temperature options	4
4.4 Precomputed table + interpolation	4
4.5 Optional error status	5
5. Computing extinction	5
6. Polarized dust optics	6
6.1 Polarized emission	6
6.2 Polarized extinction	7
6.3 Division of labor	7
6.4 Minimal example	8
6.5 Limits	8
7. The model object and accessors	9
8. The generic file loader	9
9. Linking into a 3D RT code	10
10. Units and conventions	10
11. Data files	11

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp) ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
  ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
  call dust_emission(m, J_lam, lamI_total) ! per cell
  ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*. The same populations carry the extinction-side optics, so `dust_extinction` (§5.) returns the model's opacity from that one object as well, so emission and extinction both reach the transfer code through calls, not files.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

From `SEDust/sed/`:

```
make libsedust.a # the library archive (link this into your RT code)
```

The compiler is `gfortran` with `-O3 -march=native -fopenmp` (edit the Makefile for `ifort`). `libsedust.a` is a static archive of all library objects; the Fortran `.mod` files are written alongside it in `SEDust/sed/`.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model's output channels (Table 1).

Signatures:

builder	model	channels	notes
build_astrodust	HD23 astrodust+PAH	AD, PAH	$N_C=417$, D16 graphite
build_dl07	Draine & Li 2007	SIL, CARB	$N_C=470$, D03 graphite
build_zubko	Zubko (ZDA 2004) BARE-GR-S	PAH, GRA, SIL	neutral PAH only
build_from_files	file-defined	CH1, CH2, ...	generic loader

Table 1: Built-in model builders. Each sets the model’s own optics/abundance parameters internally.

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi [, status])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi [, status])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status])
```

- NT, T_lo, T_hi: number and range [K] of the internal temperature grid (typical 200, 2.7, 5000).
- sd_index (DL07): WD01 size-distribution index — 7 = MW $R_V=3.1$, $q_{\text{PAH}}=4.58\%$; 29/30/31 = LMC2; 32 = SMC.
- U (DL07): ISRF intensity scaling (the DL07 reference uses $U=1$).
- data_dir (Zubko / files): directory holding the data files, e.g. data/zubko/.
- status (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- J_lam(m%NLAM): the local *mean intensity* on the grid m%lam [μm]. For a scaled Mathis ISRF use call J_Mathis(U, m%lam, J_lam) from module radfield.
- lamI_total(m%NLAM): the summed SED, $\lambda I_\lambda / N_H$ [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$] — the HD23 convention.
- lamI_chan(m%NLAM, m%n_channel) (optional): the SED of each channel.
- status (optional): error report; see §4.5.

4.2 Solver choice (m%stoch_method)

The field m%stoch_method selects how each grain is heated (Table 2); dust_emission honors it.

m%stoch_method	method	note
'heuristic'	GD look-ahead T-window narrowing	default ; fastest, serial
'draine'	Guhathakurta & Draine (1989)	reference GD
'qm'	energy-space transition matrix (thermal-discrete)	most detailed
'equil'	single-grain equilibrium (no stochastic)	see §4.3

Table 2: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); 'equil' forces equilibrium.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature T_{eq} , computed from *that grain's* absorption cross section, and emits $C_{\text{abs}} B_{\lambda}(T_{\text{eq}})$. No stochastic excursions are computed.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C_{\text{abs}}^{\text{tot}}(\lambda) = \sum_{\text{pop}} \sum_a \text{dn}(a) C_{\text{abs}}(\lambda, a)$. The single T_{eq} satisfies $\int C_{\text{abs}}^{\text{tot}} J d\lambda = \int C_{\text{abs}}^{\text{tot}} B(T_{\text{eq}}) d\lambda$, and the emission is $C_{\text{abs}}^{\text{tot}} B_{\lambda}(T_{\text{eq}})$. The optional `Teq_out` returns that temperature.

Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U , precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

`J_ref(m%NLAM)` is the reference field shape (at $U=1$) and `U_grid(:)` an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U . The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final status (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times (\text{fixed } J_{\text{ref}})$. For cells whose field *shape* departs from J_{ref} (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final status argument (`integer, intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints a message and stops the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply `0 = built`, nonzero = build failed.

- `dust_emission`: 1 unknown `stoch_method`; 2 'qm' selected but a population's radii are unset.
- `dust_build_table`: 1 `size(J_ref) ≠ m%NLAM`; 2 `size(U_grid) < 2`; 3 `U_grid` not positive and strictly increasing.
- `dust_emission_interp`: 1 $U \leq 0$; 2 `size(lamI_total) ≠ tab%NLAM`; 3 `lamI_chan` present but not (`tab%NLAM`, `tab%n_channel`).
- `build_astrodust`, `build_dl07`: 1 *Q*-table load failed; 2 size-distribution load failed.
- `build_zubko`: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed.
- `build_from_files`: 1 descriptor open; 2 too many pop: lines; 3 invalid channel; 4 no pop: lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed.
- `dust_extinction`: 1 an output array is not of size `m%NLAM`.

5. Computing extinction

`dust_extinction` is the extinction counterpart of `dust_emission`: a transfer code takes its opacity from the same model object, and on the same wavelength grid `m%lam`, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, Cpol_ext] [, status])
```

- `Cext`, `Cabs`, `Csca` (`m%NLAM`): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}$.
- `gbar` (`m%NLAM`) (optional): the scattering-weighted asymmetry $\langle \cos \theta \rangle$, and 0 where nothing scatters.
- `Cpol_ext` (`m%NLAM`) (optional): the dichroic (polarized) extinction [cm^2/H]; see §6..
- `status` (optional): error report; see §4.5.

Each output is the plain binned sum over every population of the model, since $dn(a)$ already carries the bin width:

$$\begin{aligned}
 C_{\text{abs}}(\lambda) &= \sum_{\text{pop}} \sum_a dn(a) C_{\text{abs}}(\lambda, a), & C_{\text{sca}}(\lambda) &= \sum_{\text{pop}} \sum_a dn(a) C_{\text{sca}}(\lambda, a), \\
 \langle \cos \theta \rangle &= \frac{\sum_a dn C_{\text{sca}} g}{\sum_a dn C_{\text{sca}}}, & C_{\text{pol,ext}}(\lambda) &= \sum_{\text{pop}} \sum_a dn(a) C_{\text{pol,ext}}(\lambda, a) f_{\text{align}}(a).
 \end{aligned} \tag{1}$$

A population that carries no scattering or polarized optics contributes zero to those terms. In the astro dust model the PAHs are exactly that case: they enter through absorption only, and the astro dust grains carry all the scattering and the asymmetry.

The values agree with the corresponding columns of `data/kext_astro dust_MW.dat` (Table 3) to the precision with which that file is written (a median relative deviation of $\sim 7 \times 10^{-9}$ in each cross section), because both evaluate the same size integral over the same optics. The difference is that the accessor returns the result on `m%lam` directly, so a host using its own wavelength grid need not interpolate off the grid the driver happened to write.

Astro dust only for scattering. `build_dl07`, `build_zubko` and `build_from_files` do not populate the extinction-side optics, so for those models `Csca`, `gbar` and `Cpol_ext` come back as zeros while `Cabs` (and hence `Cext`) is still the correct absorption sum.

6. Polarized dust optics

The library exports two polarized quantities for the HD23 astro dust model: a polarized *emission* spectrum through `dust_emission`, and a polarized *extinction* cross section through `dust_extinction` or, equivalently, through the pre-computed extinction table. Both are *intrinsic*: the size integral and the alignment weighting are done, but the geometry of the local magnetic field is not applied. What remains for the calling code is set out in §6.3.

This section documents the interface only. For the alignment physics, the polarized transfer equation, and the sign and index conventions behind these quantities, see the companion report `aligned_grain_polarization.pdf` (§3 for the optical properties, §4 for alignment, §5 for transfer).

6.1 Polarized emission

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status] [, lamI_pol])
```

- `lamI_pol(m%NLAM)` (optional): the intrinsic polarized emission, with the same shape and the same units as `lamI_total`. It is the emission a population of perfectly aligned grains would radiate with their symmetry axes in the plane of the sky.
- Only populations that carry both polarized optics and an alignment efficiency contribute. PAHs are taken to be unaligned by HD23 and add zero, as does any model built without polarized optics.
- The argument is optional and comes last, so every existing call site remains valid.

The polarized channel is accumulated at every point where the total emission is accumulated, using the same grain temperature weights, so a stochastically heated grain contributes to both consistently under all four values of `m%stoch_method`.

6.2 Polarized extinction

Two routes give the same quantity. The direct one is the optional `Cpol_ext` argument of `dust_extinction` (§5.),

```
call dust_extinction(m, Cext, Cabs, Csca, gbar, Cpol_ext)
```

which returns $\sum_a dn_{Ad}(a) C_{pol,ext}(\lambda, a) f_{align}(a)$ [cm²/H] on `m%lam`. Prefer it in a code that already holds a model object: it needs no file parsing and no interpolation onto a foreign grid.

The second route is the file `data/kext_astrodust_MW.dat`, written by `calc_kext_astrodust.x`, which carries the dichroic extinction as an eighth column (Table 3). It remains the right choice for a tool that wants the tabulated optics without linking the library. Codes that read only the first seven columns are unaffected by its presence.

col.	quantity	unit	note
1	lambda	μm	wavelength grid
2	albedo		C_{sca}/C_{ext}
3	<cos>		scattering asymmetry g
4	C_{ext}/H	cm ² /H	total extinction
5	C_{abs}/H	cm ² /H	total absorption
6	C_{sca}/H	cm ² /H	total scattering
7	K_{abs}	cm ² /g	mass absorption coefficient
8	$C_{pol,ext}/H$	cm ² /H	dichroic extinction (new)

Table 3: Columns of `kext_astrodust_MW.dat`. Column 8 is $\sum_a dn_{Ad}(a) C_{pol,ext}(\lambda, a) f_{align}(a)$, the maximum polarized extinction, quoted before any geometric factor.

6.3 Division of labor

Table 4 is the part to get right. SEDust stops at the intrinsic quantity; everything that depends on where the field points is left to the transfer code, because SEDust knows nothing about the cell geometry.

Done by SEDust	Left to the calling code
Integral over the grain size distribution	$\sin^2 \gamma$, with γ the angle between the <i>local magnetic field</i> and the <i>line of sight</i> (not a sky-plane angle)
Weighting by the alignment efficiency $f_{align}(a)$	Turbulent depolarization F_{turb}
Summation over components (only <i>astrodust</i> is aligned; PAHs contribute zero)	Splitting into Q and U using the position angle ϕ of the projected field

Table 4: What is already integrated into `lamI_pol` and into `Cpol_ext` (equivalently, column 8), and what a radiative transfer host must still apply.

The observed Stokes emissivities follow from the exported quantity as

$$(j_Q, j_U) = \text{lamI_pol} \sin^2 \gamma F_{\text{turb}} (\cos 2\phi, \sin 2\phi), \quad (2)$$

with $j_I = \text{lamI_total}$ unchanged. The same $\sin^2 \gamma$ and F_{turb} multiply Cpol_ext (column 8) when the extinction matrix is assembled. Equation (2) is stated here as a usage rule; its derivation is in §5.1 of `aligned_grain_polarization.pdf`.

6.4 Minimal example

```
use dust_lib
use radfield, only: J_Mathis
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:), lamI_pol(:), p(:)
integer :: i

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp)
allocate(J_lam(m%NLAM), lamI_total(m%NLAM), lamI_pol(m%NLAM), p(m%NLAM))

call J_Mathis(1.0_wp, m%lam, J_lam)
call dust_emission(m, J_lam, lamI_total, lamI_pol=lamI_pol)

! intrinsic polarization fraction (before any geometric factor)
do i = 1, m%NLAM
  if (lamI_total(i) > 0.0_wp) then
    p(i) = lamI_pol(i) / lamI_total(i)
  else
    p(i) = 0.0_wp
  end if
end do
```

Multiply p by $\sin^2 \gamma F_{\text{turb}}$ to obtain the polarization fraction of a real line of sight.

6.5 Limits

- **Astrodust only.** `build_dl07` and `build_zubko` have no polarized optics. `build_dl07` releases any polarized arrays left by a previous `astrodust` build, so `lamI_pol` returns zeros rather than stale values.
- **PAHs contribute zero** ($f_{\text{align}} = 0$, an HD23 assumption). The polarized emission therefore vanishes in the mid infrared, where stochastically heated PAHs dominate.
- **No circular polarization.** The released optics table carries Q_{ext} , Q_{abs} and Q_{sca} but no phase retardation, so C_{circ} cannot be formed and must be set to zero.

- **No scattering by aligned grains.** The release gives the integrated Q_{sca} with no angular information, so a Mueller matrix for an aligned spheroid cannot be built from it.
- **Far-infrared and submillimeter polarized emission is complete without either.** The albedo there is essentially zero, so the transfer needs only the extinction matrix and the emission vector, both fully determined by what is exported here.

Reference numbers. For a Mathis ISRF the intrinsic polarization fraction $\text{lamI_pol}/\text{lamI_total}$ is 0.00% at $12\mu\text{m}$, 13.67% at $100\mu\text{m}$, 17.15% at $154\mu\text{m}$, 18.79% at $350\mu\text{m}$ and 19.15% at $850\mu\text{m}$. Column 8 reproduces the released `polarized_extinction.dat` to a median of 0.0296%.

7. The model object and accessors

```
type :: dust_model_t
  character(len=32) :: name
  integer :: NA, NLAM, NT
  real(wp), allocatable :: lam(:) ! (NLAM) [um]
  real(wp), allocatable :: T_first(:) ! (NT) [K]
  type(grain_pop_t), allocatable :: pops(:)
  integer :: n_channel
  character(len=16), allocatable :: channel_name(:)
  logical :: use_induced_emission ! default .false.
  character(len=16) :: stoch_method ! default 'heuristic'
  logical :: verbose ! default .false.
end type
```

Diagnostics. `m%verbose` (default `.false.`) keeps the library solve path silent; set it `.true.` to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in `dust_lib`):

```
n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic
```

8. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One pop: line per population:

```
# descriptor.txt
name = mymodel
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat SzDist_PAH.dat Graphitic_Calorimetry.dat 0.0
pop: gra 2 Gra_121_1201.dat SzDist_Gra.dat Graphitic_Calorimetry.dat 0.0
pop: sil 3 suvSil_121_1201.dat SzDist_Sil.dat Silicate_Calorimetry.dat 0.0
```

- grain_type: 'sil', 'pah', or 'gra' (selects the heat-capacity/mode model used by the 'qm' solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- channel: integer output channel (1...); populations sharing a channel are summed.
- optics: a DustEM/Zubko Q -table (Q_{abs} per radius and wavelength); $C_{\text{abs}} = Q_{\text{abs}} \pi a^2$.
- dnda_table: a 2-column table a [μm] and dn/da [$\text{cm}^{-1} \text{H}^{-1}$].
- calorimetry: a specific-enthalpy table $u(T)$ [erg/g]; $H(T, a) = u(T) \rho \frac{4\pi}{3} a^3$.
- rho: grain density [g cm^{-3}]; 0.0 uses the value in the optics-file header.

All files are sought under data_dir. The coded builders (build_astrodust/dl07/zubko) are the recommended path for the built-in models; build_from_files is for adding new data-defined models.

9. Linking into a 3D RT code

Compile your RT source against libsedust.a with the .mod search path pointing at SEDust/sed/:

```
gfortran -I<sed> my_rt.f90 <sed>/libsedust.a -fopenmp -o my_rt.x
```

A complete minimal example is provided in SEDust/sed/rt_example/use_dustlib.f90: it builds a model, computes emission for one field, and prints the SED peak. No ISO_C_BINDING is needed — the RT code simply uses the Fortran module dust_lib.

Threading. m is read-only during dust_emission; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default 'heuristic' solver is serial per call, which suits an outer cell-level parallel loop.

10. Units and conventions

- Wavelengths λ are in μm throughout; cross sections in cm^2 .
- J_{λ} is a mean intensity with the same units as the Planck function B_{λ} (specific intensity per steradian). The library's output $\lambda I_{\lambda}/N_{\text{H}}$ is then in $\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$.

- The CMB (2.725 K) is included in the heating field by J_Mathis.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

11. Data files

Model data lives under SEDust/data/:

- astrodust / DL07: the T-matrix Q -table (`../tmatrix/output/q_astrodust_*.dat`) and the HD23 release size distribution (`../data/release/size_distribution.dat`).
- Zubko: `../data/zubko/` — the ZDA config/formula, the tabulated size distributions, the DustEM Q -tables, and the calorimetry tables from the Camps et al. 2015 benchmark; a ready descriptor is `../data/zubko/zubko_descriptor.txt`.